

Aprendizaje Automático

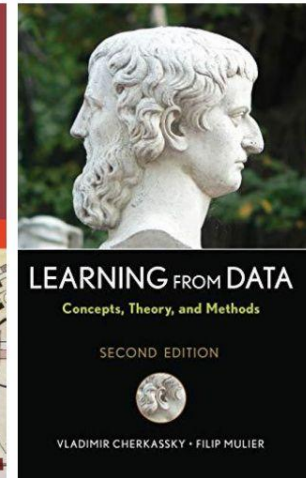
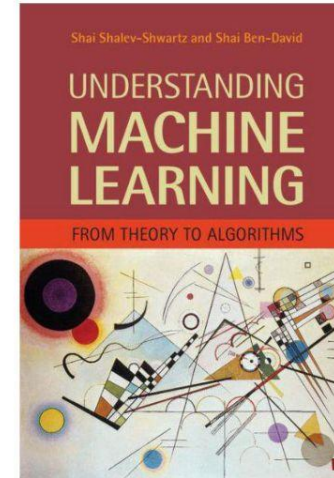
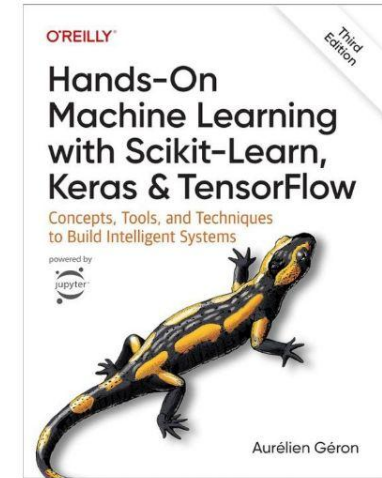
Curso 2024-2025

Tema 5. Aprendizaje Profundo. Fundamentos

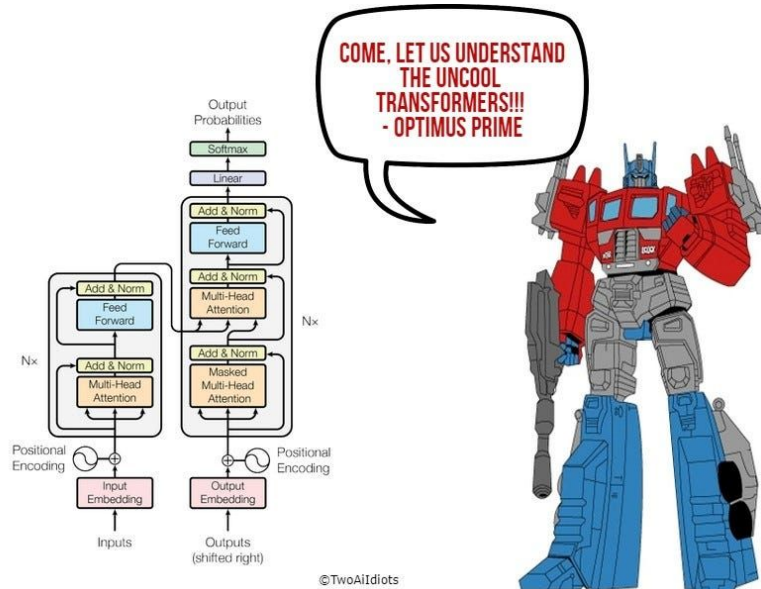
Parte II: Modelos Transformer



Bibliografía



Modelos basados en atención: Transformer



UNIVERSIDAD
DE GRANADA



Instituto Andaluz Interuniversitario en
Data Science and Computational Intelligence

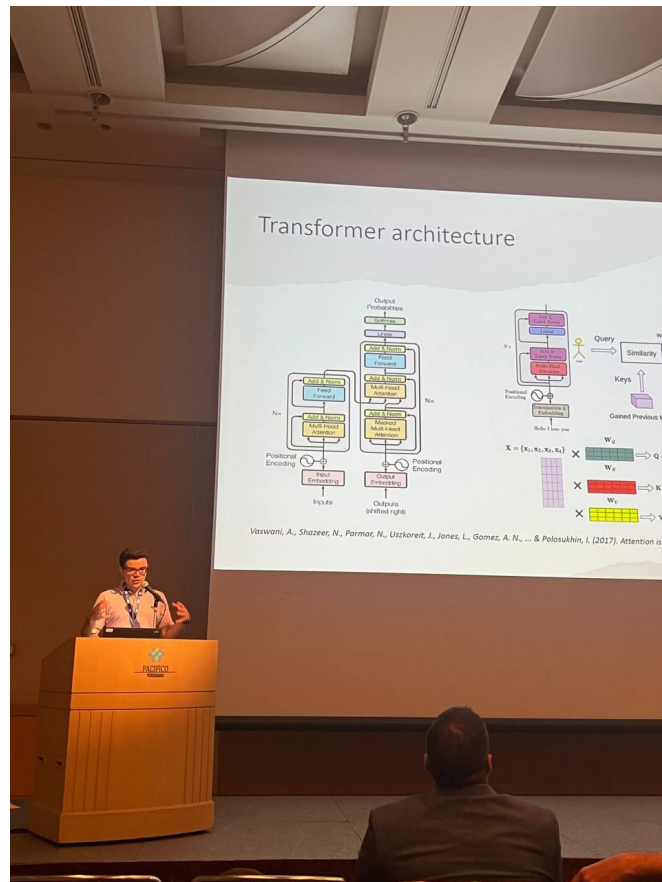
Ignacio Aguilera Martos

nacheteam@ugr.es

¿Quién soy yo?

- Doble Grado Matemáticas e Informática UGR
- Master en ciencia de datos
- Doctor en IA
- Background en anomalías, series temporales, redes neuronales recurrentes y Transformers.

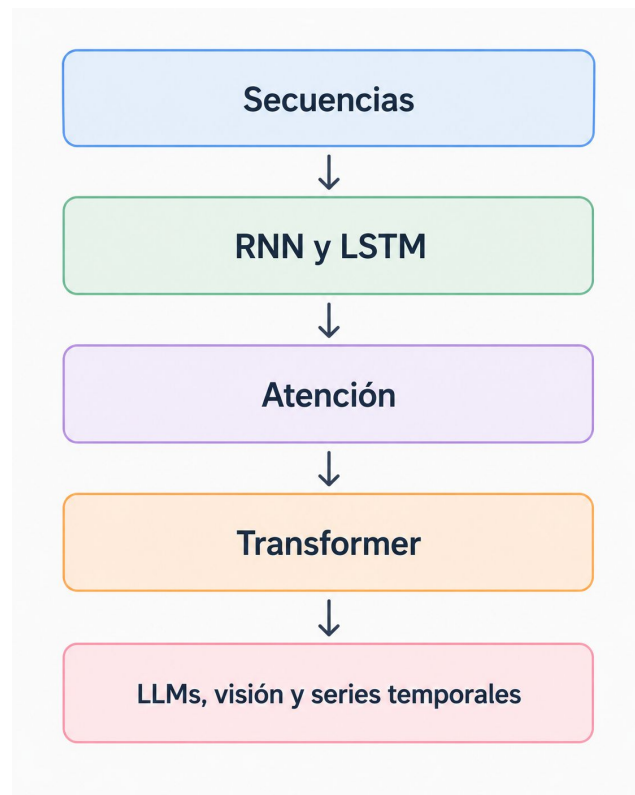
nacheteam@ugr.es



Objetivos de la clase

Al final de la clase deberías saber:

- ¿Por qué las secuencias son difíciles de modelar?
- ¿Qué limitaciones tienen las redes recurrentes?
- ¿Que es el mecanismo de atención?
- ¿Cómo se construye un bloque Transformer?
- ¿Por qué los Transformers son la base de muchos modelos actuales?



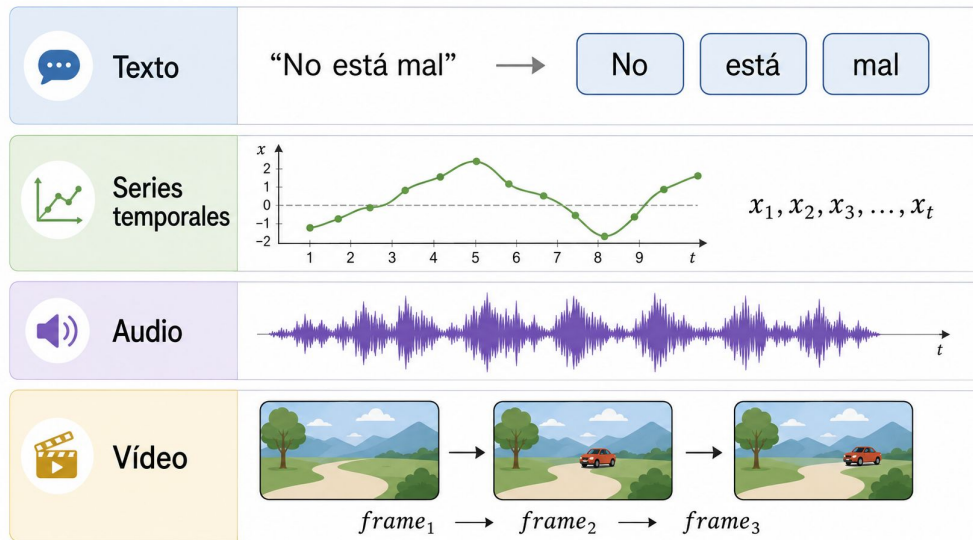
Índice

- 1. Secuencias, redes densas, convoluciones, recurrencia y LSTM**
2. Intuiciones y definición de Atención
3. Cross-Attention, Self-Attention y módulo Transformer
4. Arquitectura y elementos complementarios
5. Aplicaciones

Secuencias

Secuencias:

- Texto: “La película no está mal”
- Series temporales: temperatura, consumo eléctrico, tráfico, bolsa, ...
- Audio: señal cambiante a lo largo del tiempo
- Video: secuencia de imágenes



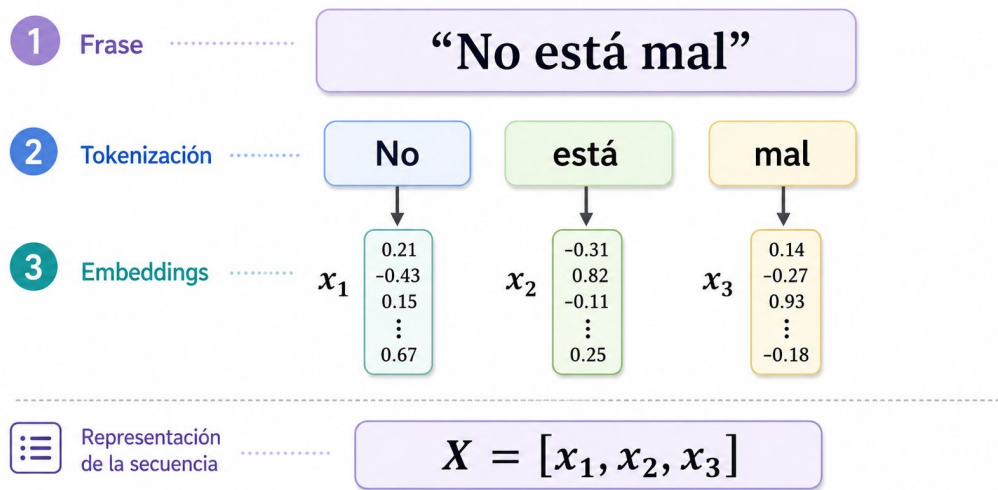
Secuencia = elementos + orden + contexto

¿Cómo ve una red neuronal una secuencia?

Una red neuronal no procesa palabras, sonidos o imágenes directamente.

Primero convertimos cada elemento en un vector:

- palabra -> embedding
- valor temporal -> vector de características
- frame de video -> representación visual



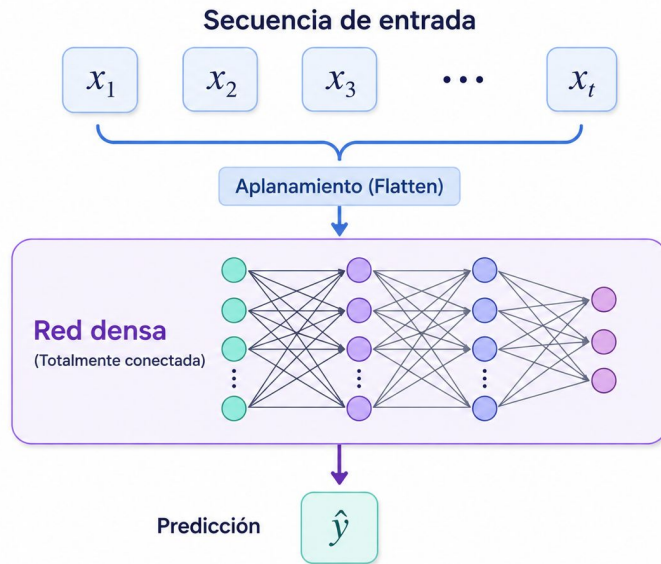
Secuencia -> vectores -> matriz de entrada

¿Y si usamos una red densa?

Podemos transformar la secuencia en un vector y pasarlo por una red densa.

Problemas:

- Trata la entrada como un bloque fijo.
- No aprovecha el orden temporal.
- No generaliza a secuencias de longitud variable.
- Las relaciones lejanas y cercanas se mezclan.



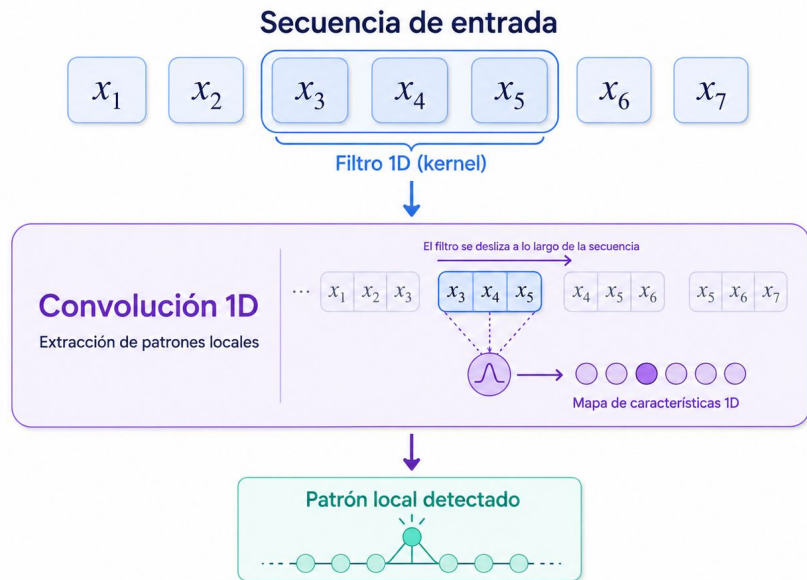
Necesitamos una arquitectura que entienda el orden de los datos.

Otra idea: mirar los patrones locales

Las redes convolucionales aprenden filtros locales.

En imágenes:
bordes -> formas -> objetos

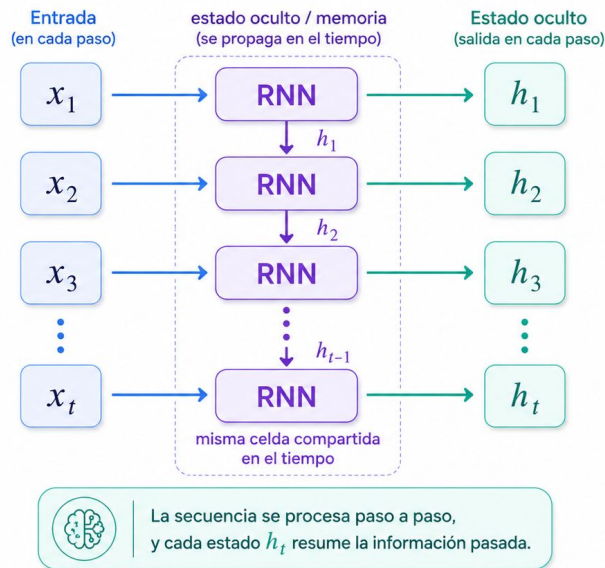
En series o audio:
cambios locales -> patrones
temporales -> eventos



Una CNN entiende bien la localidad, pero no siempre captura dependencias largas.

Redes recurrentes: procesar paso a paso

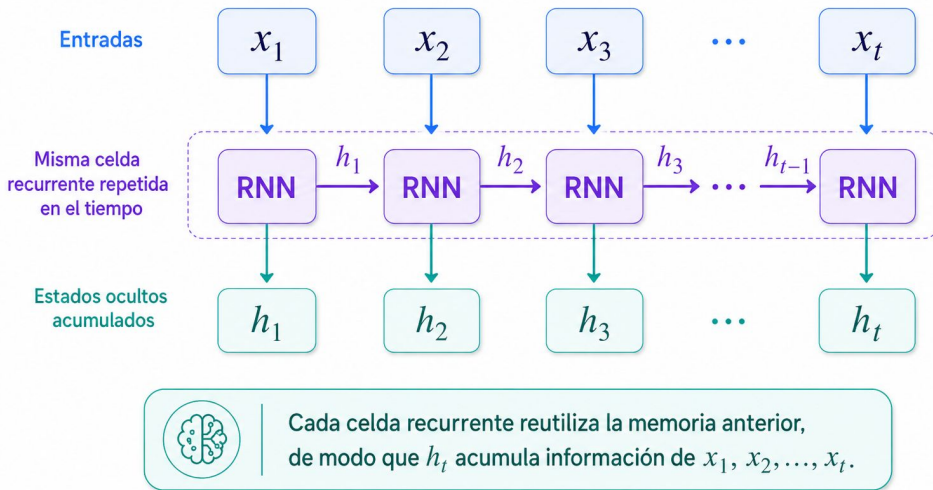
Procesamos la secuencia elemento a elemento, manteniendo una memoria interna. Cada estado oculto resume lo que la red ha visto hasta ese momento.



Una RNN convierte una secuencia en una memoria que se actualiza con cada nuevo elemento.

La misma red, repetida muchas veces

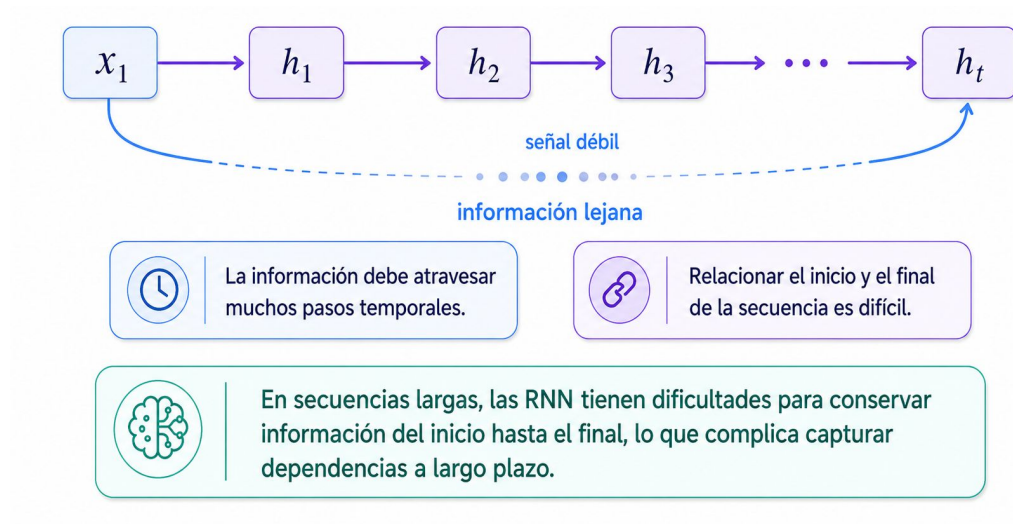
Una RNN aplica la misma celda en cada instante. La memoria viaja por la secuencia a través del estado oculto.



Una RNN convierte una secuencia en una memoria que se actualiza con cada nuevo elemento.

El problema: recordar información lejana

En una RNN, la información viaja paso a paso pero si la secuencia es larga, la información inicial puede perderse.

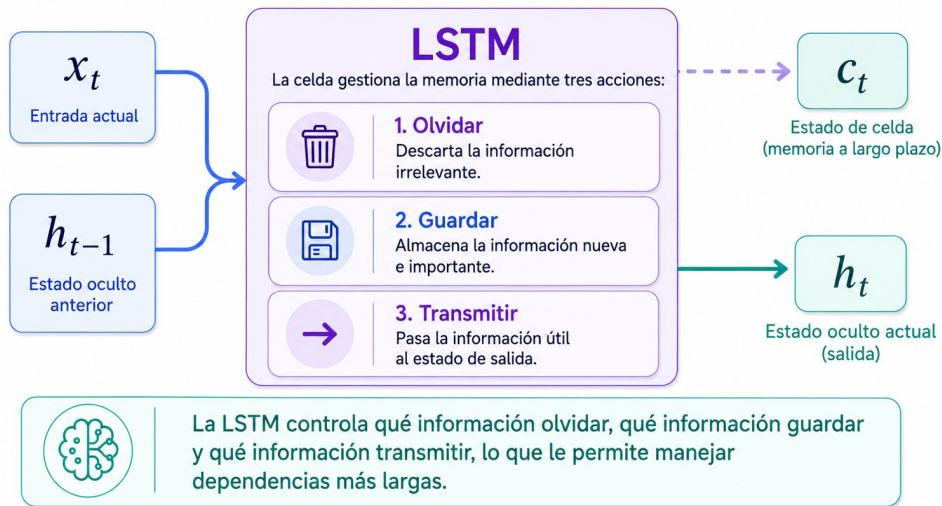


Cuanto más larga es la secuencia, más difícil es mantener dependencias a largo plazo.

LSTM: decidir qué recordar y qué olvidar

Las LSTM mejoran las RNN añadiendo mecanismos de control:

- Puerta de olvido: qué información descartamos.
- Puerta de entrada: qué información nueva guardamos.
- Puerta de salida: qué parte de la memoria usamos.

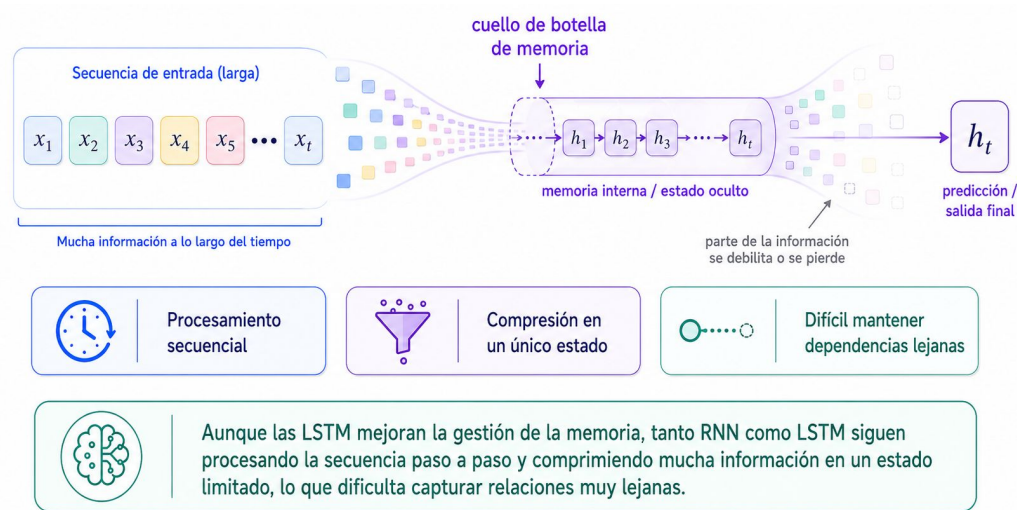


Una LSTM no sólo actualiza la memoria: aprende a regularla

El límite: seguimos procesando en cadena

Aunque las LSTM mejoran la memoria, mantienen varios problemas:

- Procesamiento secuencial: no podemos paralelizar bien.
- Dependencias largas: la información sigue viajando paso a paso.
- Cuello de botella: todo debe pasar por el estado oculto.
- Entrenamiento muy lento



Necesitamos que los elementos de la secuencia puedan comunicarse directamente

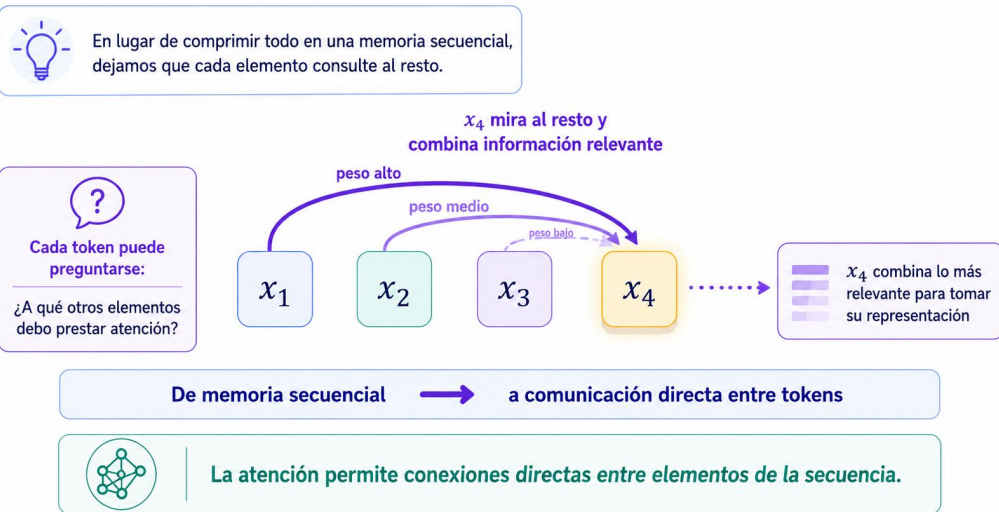
Index

1. Secuencias, redes densas, convoluciones, recurrencia y LSTM
- 2. Intuiciones y definición de Atención**
3. Cross-Attention, Self-Attention y módulo Transformer
4. Arquitectura y elementos complementarios
5. Aplicaciones

Atención: mirar directamente lo importante

En lugar de comprimir todo en una memoria secuencial, dejamos que cada elemento consulte al resto.

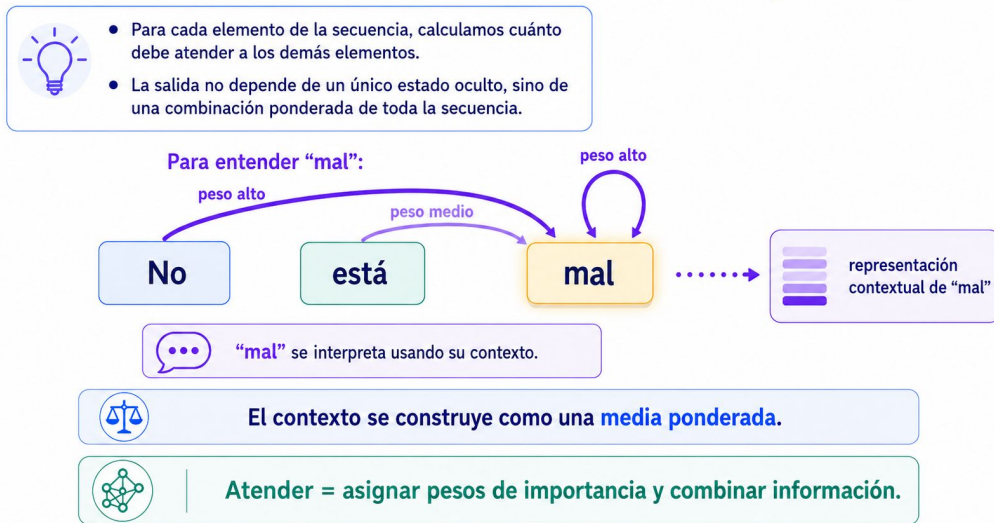
Cada token puede preguntarse: ¿A qué otros elementos debo prestar atención?



La atención permite conexiones directas entre elementos de la secuencia

Atención: no toda la información importa igual

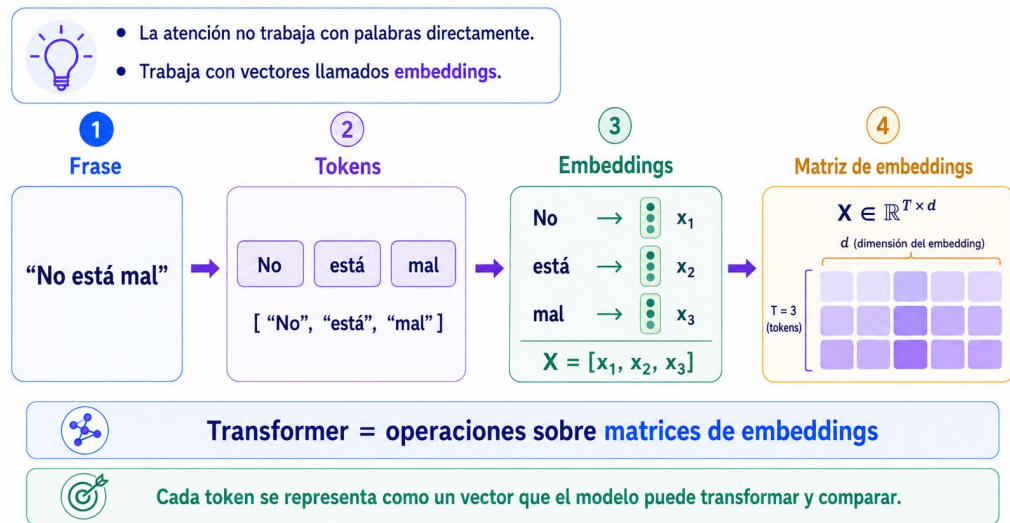
Para cada elemento de la secuencia, calculamos cuánto se debe atender a los demás elementos. La salida no depende de un único estado oculto, sino de una combinación ponderada de toda la secuencia.



Atender = asignar pesos de importancia y combinar la información

Antes de atender: convertir elementos en vectores

La atención no trabaja con palabras directamente. Trabaja con vectores llamados embeddings.



Cada token se representa como un vector que el modelo puede transformar y comparar

Atención: preguntar, comparar y recoger información

A partir de cada embedding vamos a generar tres vectores:

- Query (Q): qué estoy buscando
- Key (K): qué información ofrezco
- Value (V): qué información transmito

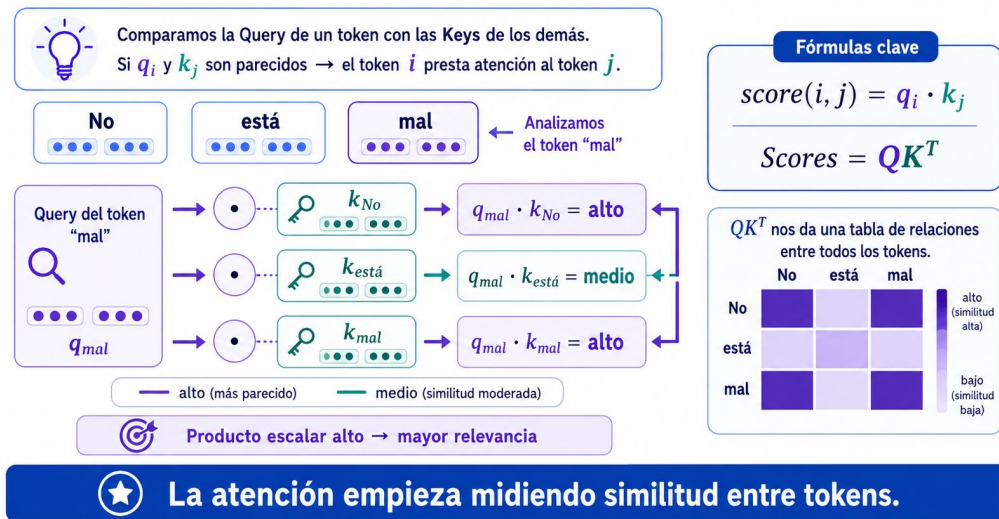
Cada token hace una pregunta-> ¿Qué otros tokens son relevantes para mí?
 Compara su Query con las Keys de los demás, después combina sus Values según la relevancia obtenida.



Q y K deciden a quién miro. V contiene la información que recojo.

¿A quién debe mirar cada token?

Comparamos la Query de un token con las Keys de los demás. Si q_i y q_j son parecidos, entonces el token i presta atención al token j .



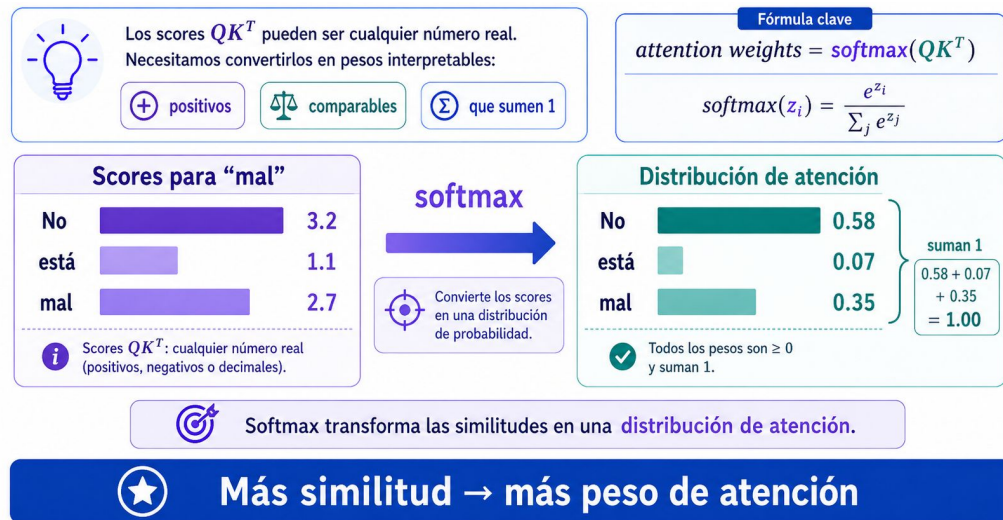
QK^T nos da una tabla de relaciones entre todos los tokens

Convertir similitudes en pesos de atención

Los scores QK^T pueden ser cualquier número real. Necesitamos convertirlos en pesos interpretables:

- positivos
- comparables
- que sumen 1

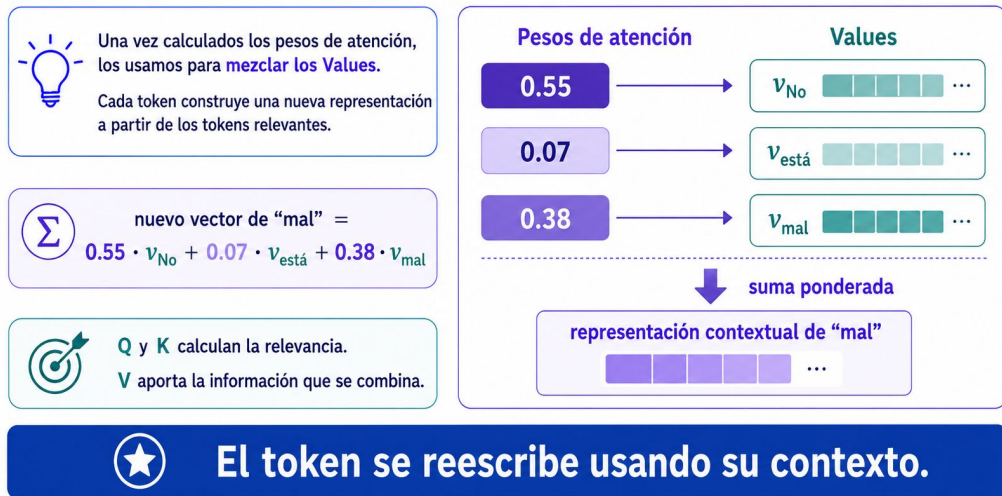
$$attention\ weights = softmax(QK^T)$$



Softmax transforma las similitudes en una distribución de atención.

Atender también es combinar información

Una vez calculados los pesos de atención, los usamos para mezclar los Value. Cada token construye una nueva representación a partir de los tokens relevantes.



Q y K calculan la relevancia. V aporta la información que se combina.

Scaled Dot-Product Attention

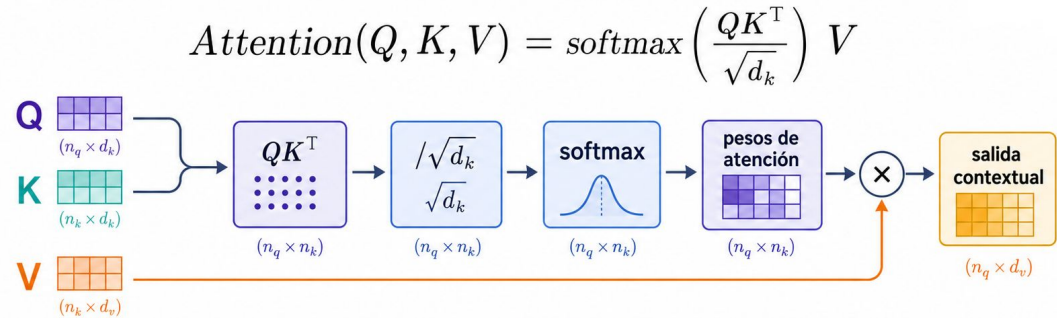
$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$QK^T \rightarrow$ compara queries con keys

$/\sqrt{d_k} \rightarrow$ estabiliza la escala

$softmax \rightarrow$ convierte scores en probabilidades

$V \rightarrow$ mezcla la información



La atención compara, normaliza y combina.

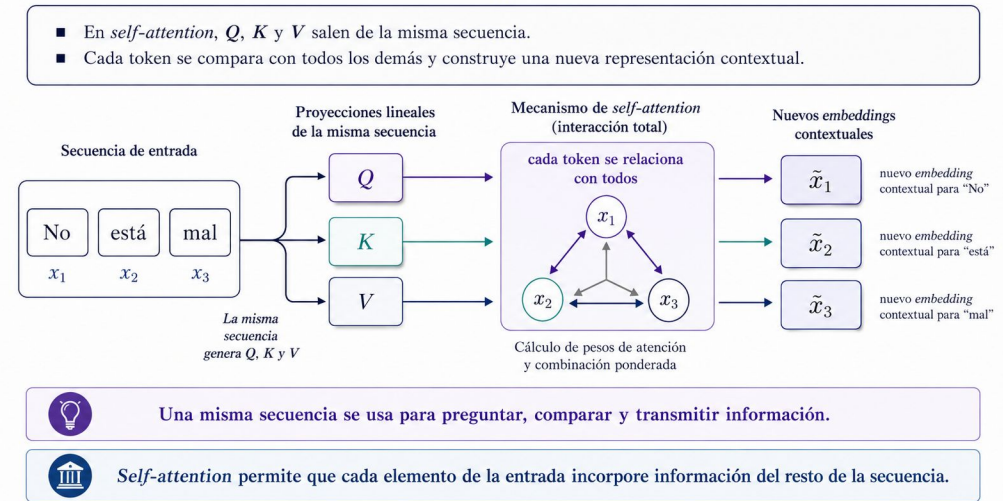
La atención compara, normaliza y combina.

Index

1. Secuencias, redes densas, convoluciones, recurrencia y LSTM
2. Intuiciones y definición de Atención
- 3. Cross-Attention, Self-Attention y módulo Transformer**
4. Arquitectura y elementos complementarios
5. Aplicaciones

Self-attention: los tokens se comparan entre sí

En self-attention Q , K y V salen de la misma secuencia. Cada token se compara con todos los demás y construye una nueva representación contextual. Self-attention permite prestar atención a los elementos de la secuencia para relacionarlos entre sí y obtener dependencias a largo plazo y relaciones directas entre tokens.

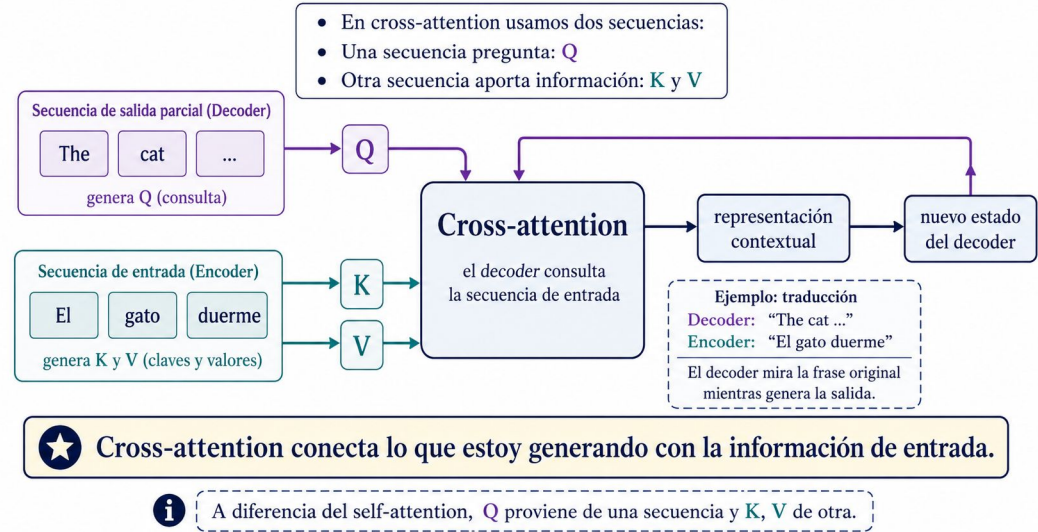


Self-attention permite que cada elemento de la entrada incorpore información del resto de la secuencia.

Cross-attention: atender a otra secuencia

En cross-attention usamos dos secuencias:

- Una secuencia pregunta: Q
- Otra secuencia que aporta información: K y V.



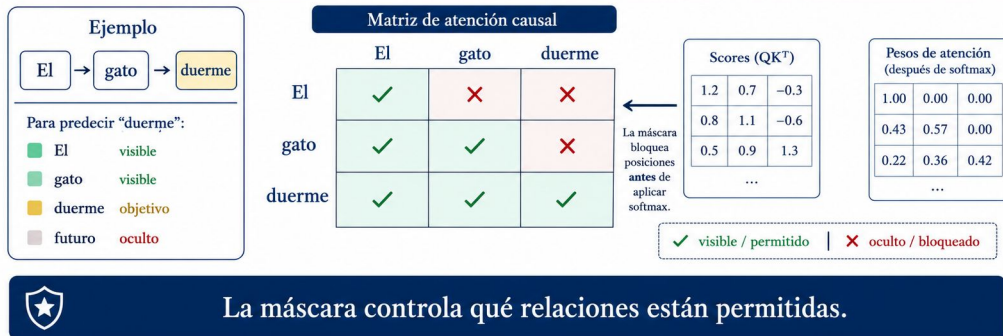
Cross-attention conecta lo que estoy generando con la información de entrada.

Máscaras de atención: impedir que el modelo haga trampas

A veces no queremos que un token mire a todos los demás. En generación de texto, cada token solo puede usar:

- Los tokens anteriores.
- El token actual.
- Nunca los tokens futuros.

- A veces no queremos que un token mire a todos los demás.
- En generación de texto, cada token solo puede usar:
 - los tokens anteriores
 - el token actual
 - nunca los tokens futuros



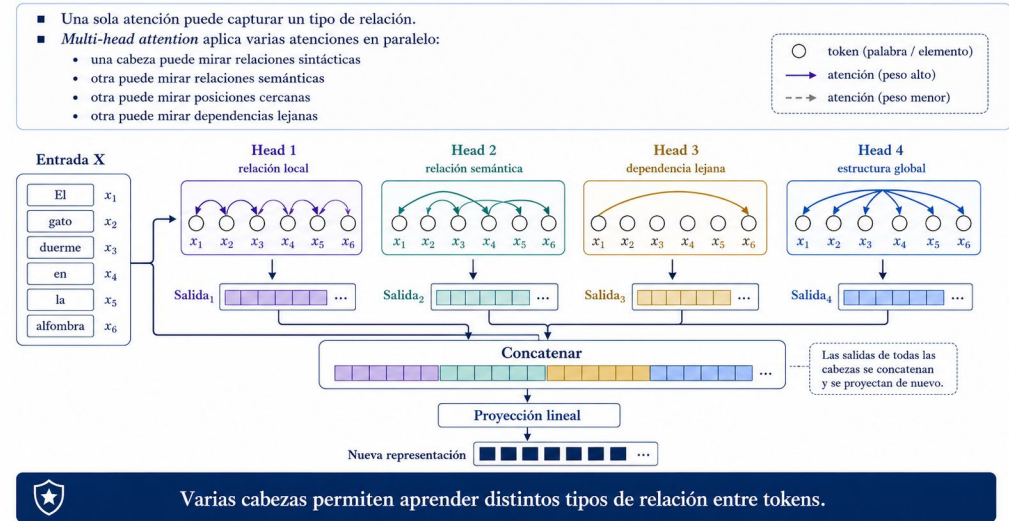
• Masked attention = atención con reglas •

La máscara controla qué relaciones están permitidas.

Multi-head attention: mirar desde varias perspectivas

Una sola atención puede capturar un tipo de relación. Multi-head attention aplica varias atenciones en paralelo:

- Una cabeza puede mirar relaciones sintácticas
- Otra puede mirar relaciones semánticas
- Otra puede mirar posiciones cercanas
- Otra puede mirar dependencias lejanas



Varias cabezas permiten aprender distintos tipos de relación entre tokens.

Index

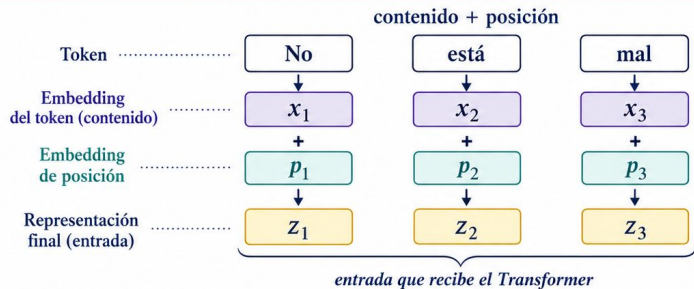
1. Secuencias, redes densas, convoluciones, recurrencia y LSTM
2. Intuiciones y definición de Atención
3. Cross-Attention y Self-Attention
- 4. Arquitectura y elementos complementarios**
5. Aplicaciones

Positional encoding: ¿dónde queda el orden?

La atención compara tokens entre sí pero no conoce su posición. Debemos añadir información posicional a cada embedding de entrada.

- La atención compara tokens entre sí,
- pero no conoce automáticamente su posición.
- Por eso añadimos información posicional
- a cada embedding de entrada.

$$\text{embedding final} = \text{embedding del token} + \text{embedding de posición}$$



En el Transformer original, la posición se codifica con funciones seno y coseno.

En otros modelos, las posiciones pueden aprenderse.



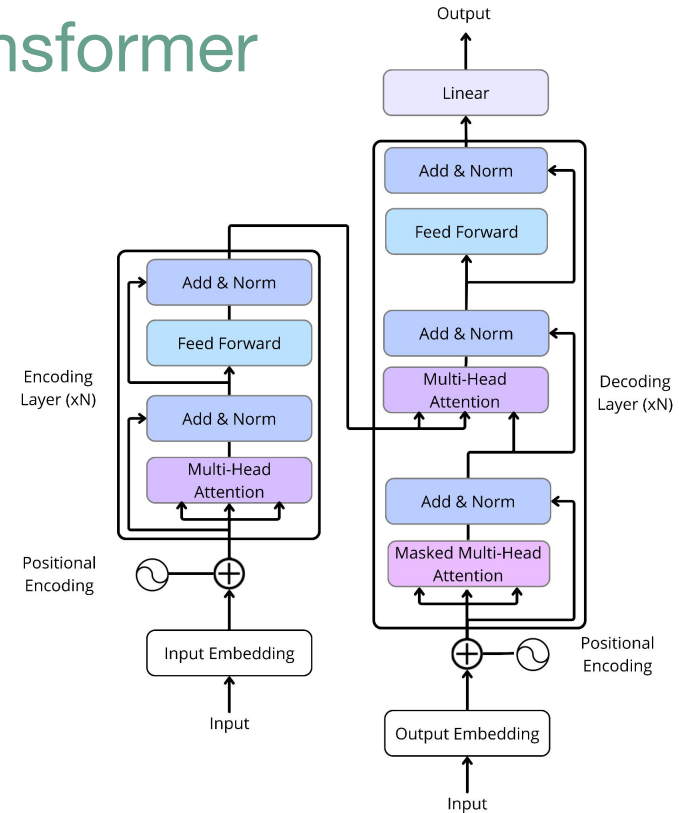
El Transformer necesita saber qué elemento va antes y cuál va después.

entrada del Transformer = contenido + posición

El bloque Transformer

Un bloque Transformer combina dos ideas:

1. Multi-head Attention: los tokens intercambian información.
2. Feed-forward network: cada token transforma su propia representación.

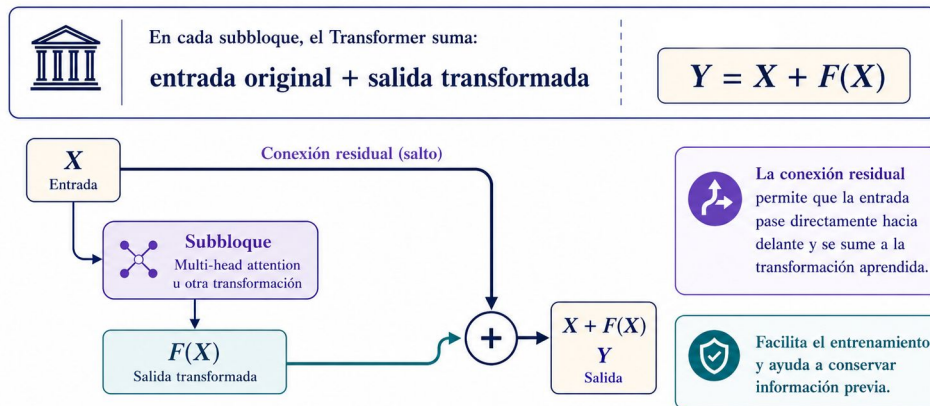


La atención mezcla la información entre tokens, la red feed-forward procesa cada token por separado.

Residual connections: no perder la información original

En cada sub-bloque el Transformer suma la entrada original a la entrada transformada. Ayuda en redes muy profundas en varios sentidos:

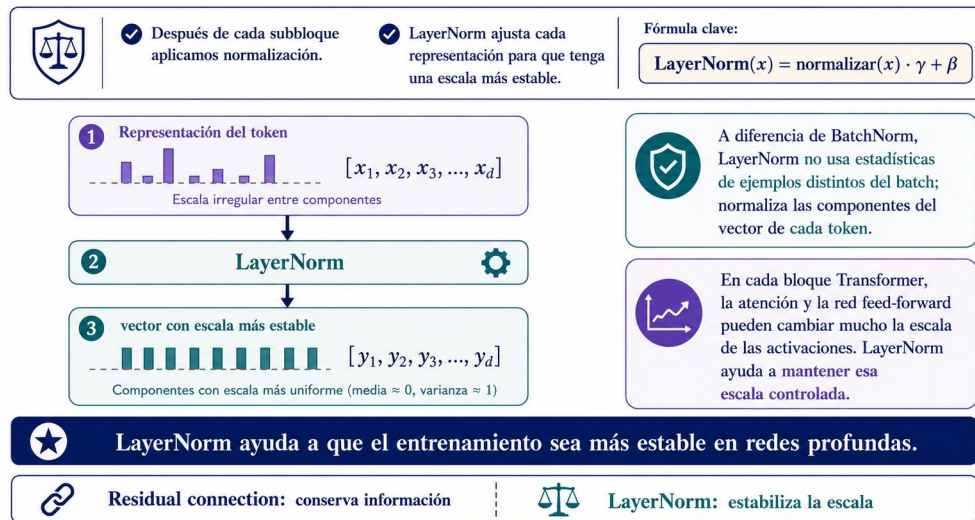
- Se pueden hacer contribuciones pequeñas acumulativas por la red.
- Ayuda a estabilizar el gradiente ya que siempre queda el camino identidad.
- Se puede interpretar como comunicación de resultados a través de un buffer.



Las conexiones residuales nos ayudan a no olvidar.

Layer Normalization: estabilizar las representaciones

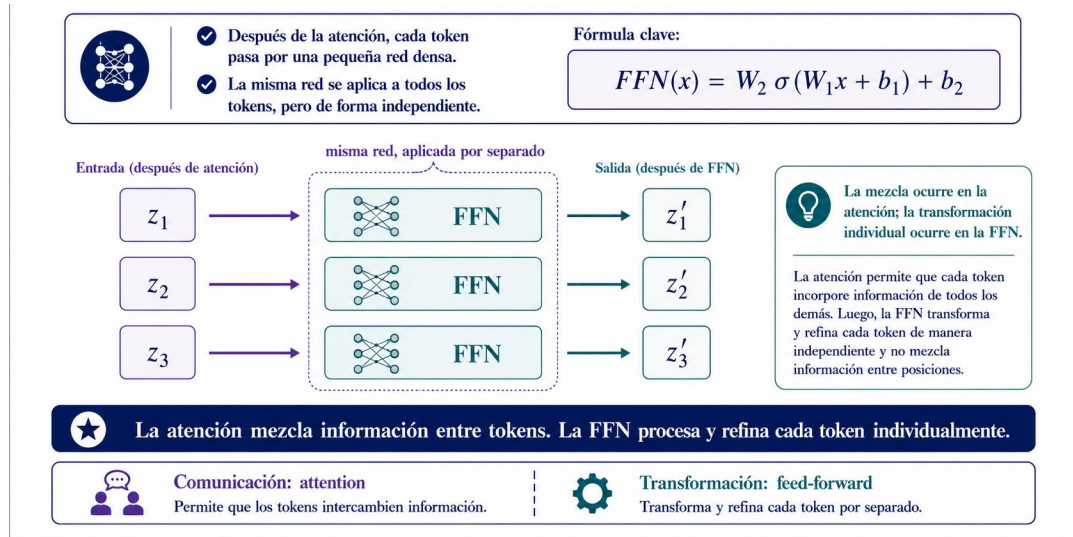
Después de cada sub-bloque aplicamos normalización. LayerNorm ajusta cada representación para que tenga una escala más estable.



Feed-forward: transformar cada token por separado

Después de la atención, cada token pasa por una pequeña red densa.

La misma red se aplica a todos los tokens, pero de forma independiente.



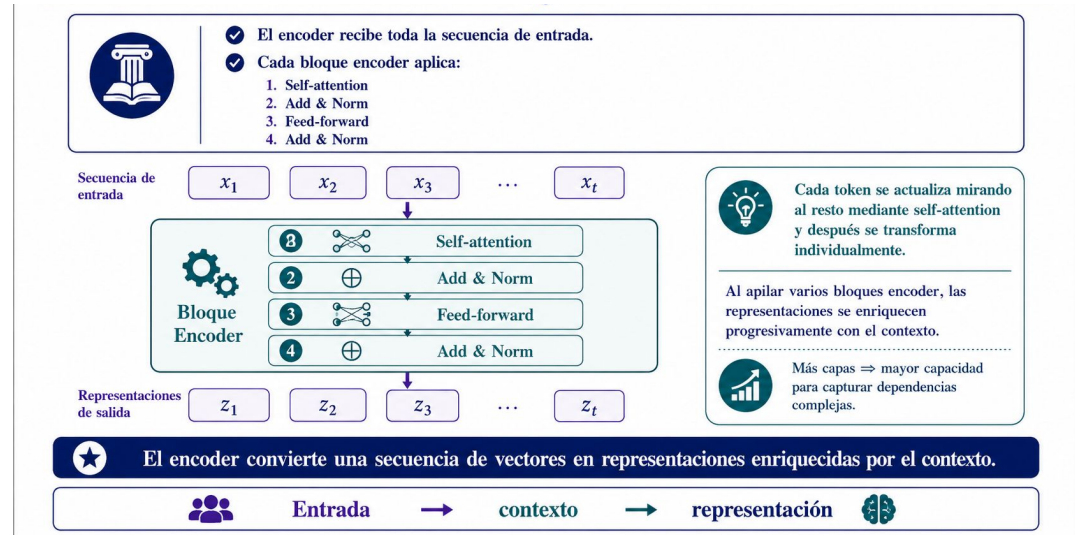
La atención mezcla información entre tokens. La FFN procesa y refina cada token individualmente.

Encoder: construir una representación contextual

El encoder recibe toda la secuencia de entrada.

Cada bloque encoder aplica:

1. Self-attention
2. Add & Norm
3. Feed-forward
4. Add & Norm



El encoder convierte una secuencia de vectores en representaciones enriquecidas por el contexto.

Decoder: generar una salida paso a paso

El decoder genera una secuencia de salida.

En cada bloque usa:

1. Masked self-attention
2. Cross-attention
3. Feed-forward network

El **decoder** genera una secuencia de salida.

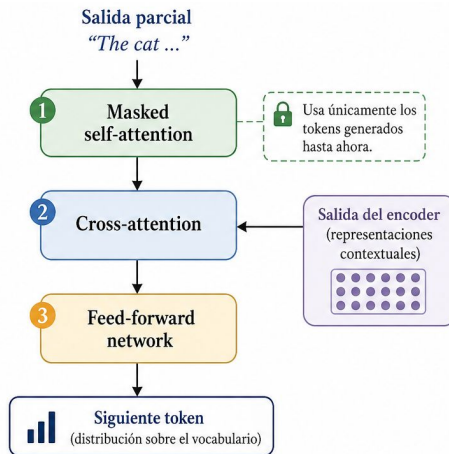
En cada bloque usa:

- 1 **Masked self-attention**
Mira solo los tokens generados hasta ahora.
- 2 **Cross-attention**
Consulta la representación producida por el encoder.
- 3 **Feed-forward network**
Transforma cada token individualmente.



Mensaje clave

El decoder combina lo que ya ha generado con la información de entrada.

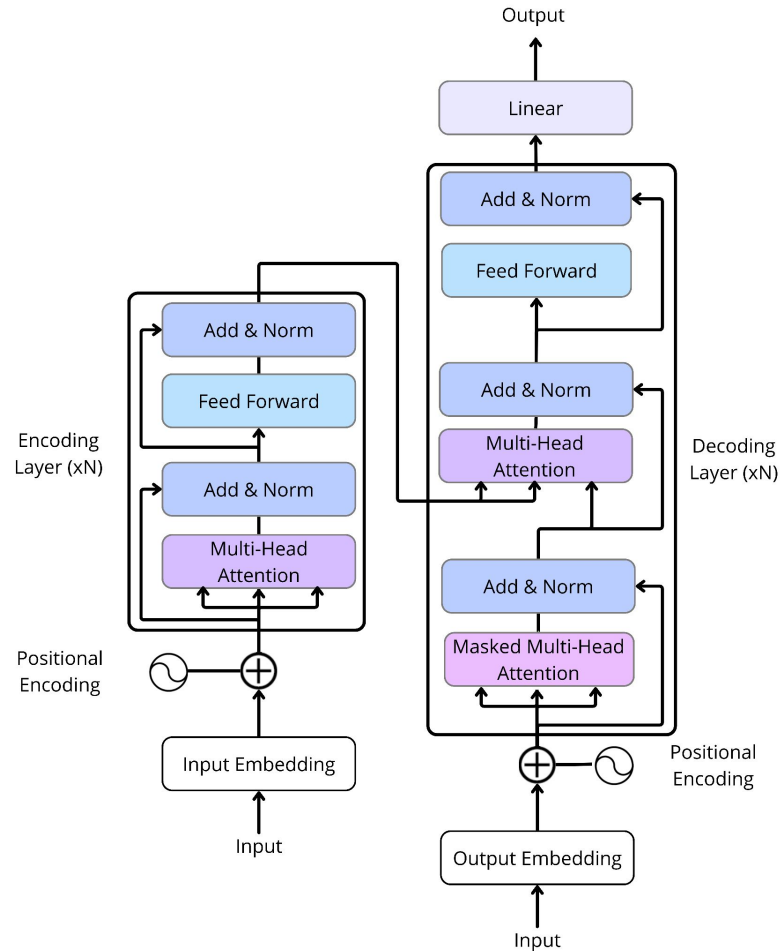


Flujo general: el decoder genera la salida paso a paso, combinando el contexto previo con la información del encoder.

El decoder combina lo que ya ha generado con la información de entrada.

Retomamos de nuevo el modelo

37

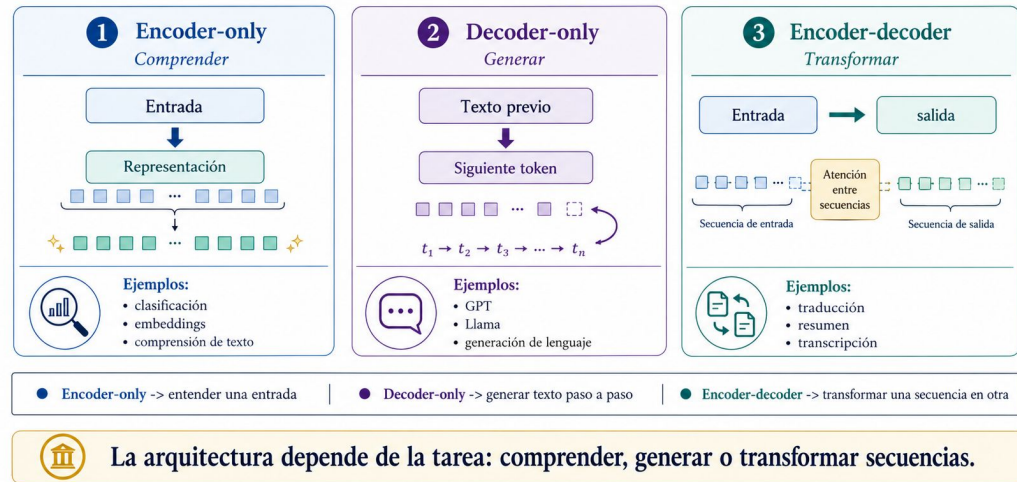


No todos los Transformers tienen la misma arquitectura

Según el problema empleamos distintas arquitecturas:

- Encoder-only: generar embeddings, comprender, comprimir información.
- Decoder-only: generación de salida, más rápido, fácil de acoplar con enriquecimiento de entrada.
- Encoder-decoder: arquitectura completa. Muchas veces se emplea en entrenamiento y luego se trocea en inferencia.

Según el problema, usamos distintas arquitecturas:

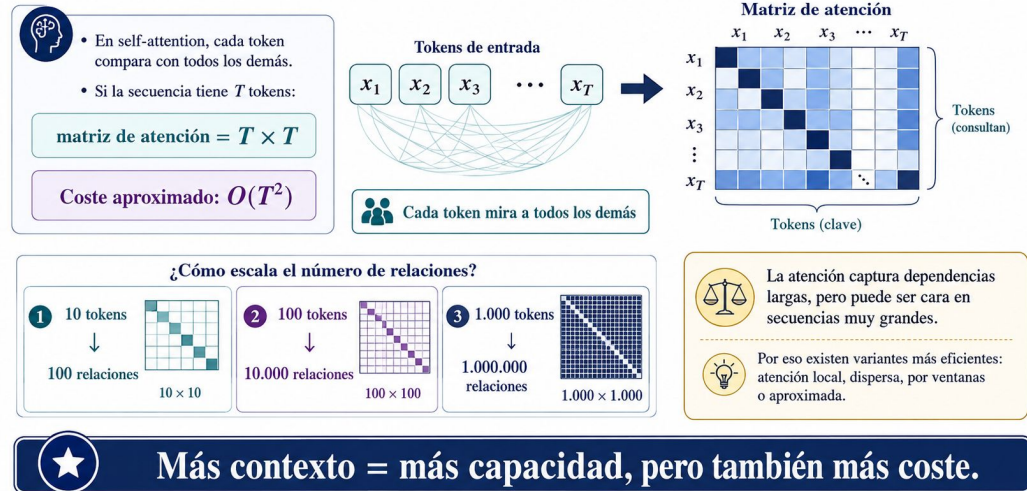


La arquitectura depende de la tarea: comprender, generar o transformar secuencias.

El precio de la atención

En self-attention cada token mira a todos los demás. Si tenemos T tokens entonces tenemos $T \times T$ relaciones que establecer. La complejidad del mecanismo de atención es

$$\Theta(T^2)$$



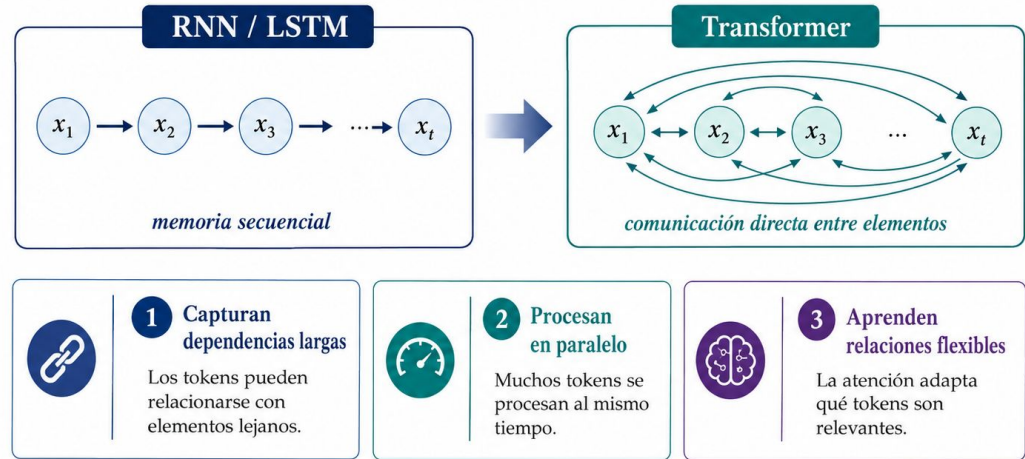
La atención captura dependencias a largo plazo, pero en grandes secuencias puede ser muy costosa.

¿Por qué han funcionado tan bien?

Los Transformers tienen tres ventajas clave:

1. Capturan dependencias a largo plazo
2. Procesan la secuencia en paralelo
3. Aprenden relaciones flexibles entre tokens

El procesamiento en paralelo es lo que ha empujado que estas arquitecturas sean mucho más grandes. No podríamos haber tenido modelos de trillones (americanos) de parámetros con LSTM.



La atención sustituye la memoria secuencial por comunicación directa entre elementos.

No son una solución mágica

Los Transformers también tienen limitaciones:

1. Coste cuadrático con la longitud de la secuencia.
2. Necesitan mucha memoria y datos.
3. No entienden el orden sin información posicional.
4. Pueden aprender correlaciones espurias.
5. Son difíciles de interpretar.

Los Transformers también tienen limitaciones:

1. Coste cuadrático con la longitud de la secuencia.
2. Necesitan mucha memoria y datos.
3. No entienden el orden sin información posicional.
4. Pueden aprender correlaciones espurias.
5. Son difíciles de interpretar.

Fortalezas		Limitaciones
 dependencias largas	→	 coste alto
 paralelización	→	 mucha memoria
 flexibilidad	→	 difícil interpretación
 gran escala	→	 dependencia de datos



Los Transformers han sido muy exitosos, pero no resuelven todos los problemas. Su capacidad para mirar toda la secuencia es potente, pero cara. Además, necesitan muchos datos, mucha memoria y una buena estrategia de entrenamiento. También pueden aprender patrones superficiales o sesgos presentes en los datos.

Más capacidad no significa necesariamente más comprensión de los datos.

Index

1. Secuencias, redes densas, convoluciones, recurrencia y LSTM
2. Intuiciones y definición de Atención
3. Cross-Attention, Self-Attention y módulo Transformer
4. Arquitectura y elementos complementarios
- 5. Aplicaciones**

Modelos actuales: no todo es un chatbot

Hoy encontramos tres niveles distintos:

1. Productos cerrados

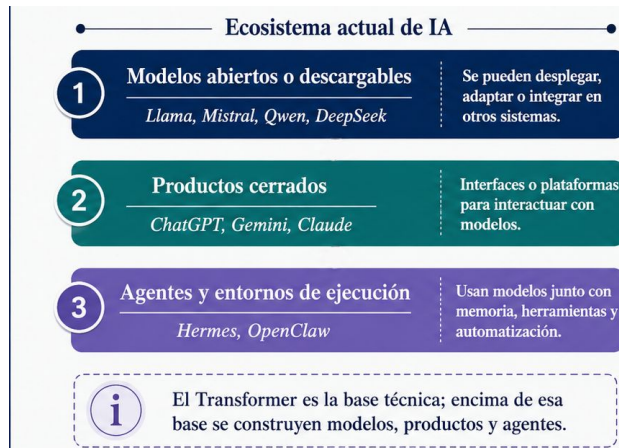
ChatGPT, Gemini, Claude

2. Modelos abiertos o descargables

Llama, Mistral, Qwen, DeepSeek

3. Agentes y entornos de ejecución

Hermes, OpenClaw



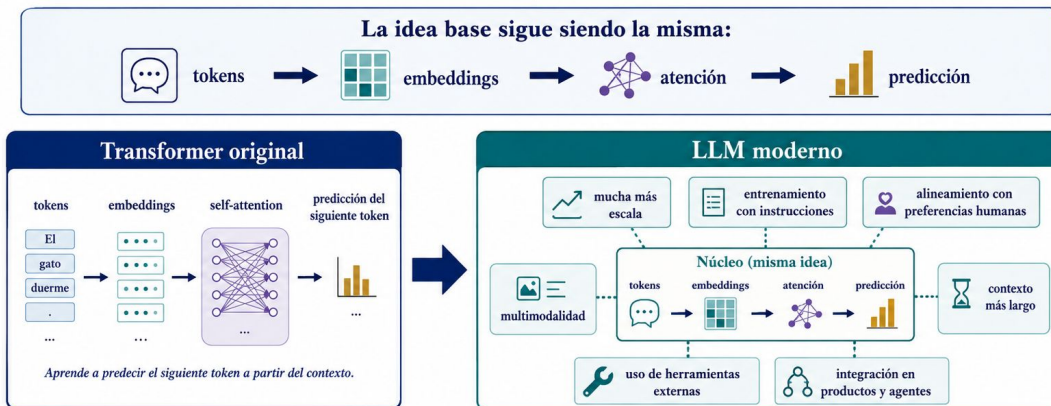
Nombre	Qué es
ChatGPT	Producto basado en modelos OpenAI
Gemini	Ecosistema de modelos/productos de Google
Claude	Familia de modelos de Anthropic
Llama	Familia de modelos abiertos de Meta
Hermes	Agente open-source con memoria y skills
OpenClaw	Asistente/agente local con herramientas

De predecir tokens a construir herramientas.

¿Qué ha cambiado respecto al transformer original?

La idea base sigue siendo la misma, pero los modelos añaden:

- Mucha más escala
- Entrenamiento con instrucciones
- Alineamiento con preferencias humanas
- Contexto más largo
- Multimodalidad
- Uso de herramientas externas
- Integración en productos y agentes



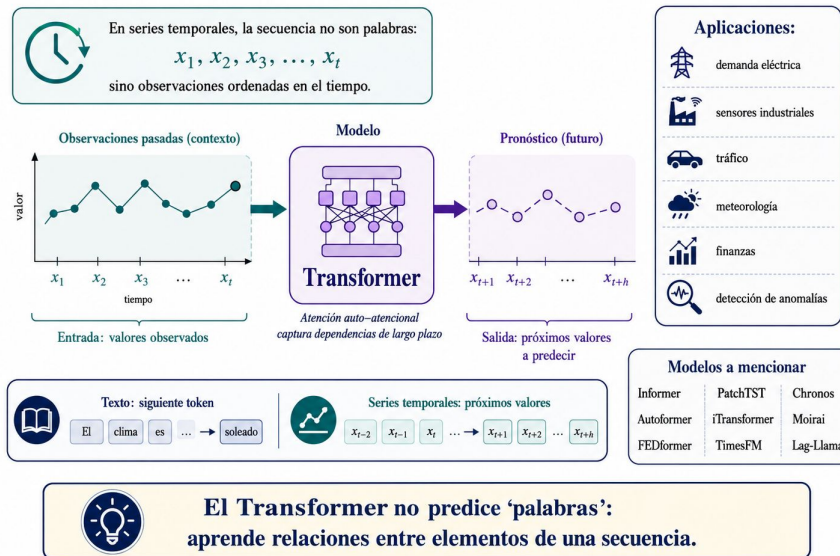
Los LLM modernos son Transformers escalados, adaptados y conectados a herramientas

Series temporales: predecir el futuro con contexto

En series temporales, las secuencias no son palabras sino observaciones ordenadas en el tiempo.

- Demanda eléctrica
- Sensores industriales
- Tráfico
- Meteorología
- Finanzas
- Detección de anomalías

Series temporales: predecir el futuro con contexto

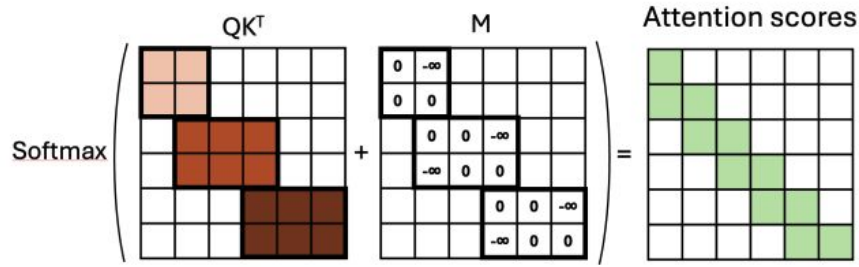


Un Transformer aprende relaciones entre elementos de una secuencia.

Series temporales: predecir el futuro con contexto

Modelos SOTA:

- Informer
- Autoformer
- FEDformer
- PatchTST
- Chronos
- Local Attention

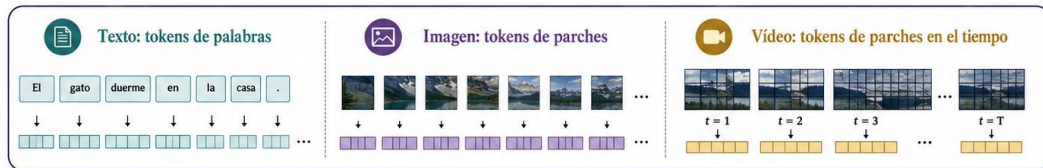
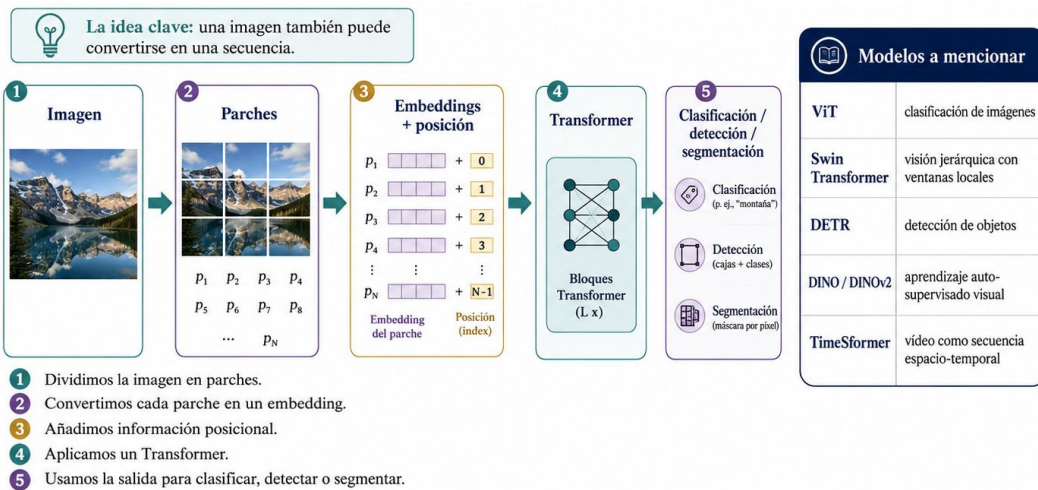


Los modelos Transformer son ampliamente usados en múltiples campos de aplicación.

Visión por computador: una imagen como secuencia

47

1. Dividimos la imagen en parches.
2. Convertimos cada parche en un embedding.
3. Añadimos información posicional.
4. Aplicamos el modelo Transformer
5. Usamos la salida para clasificar, detectar, segmentar, etc.



El Transformer no necesita que la secuencia sea texto, sólo necesita una secuencia de vectores.

Qué hemos aprendido

1. Las secuencias necesitan contexto
2. Las RNN procesan secuencialmente paso a paso.
3. La atención permite comunicación directa entre elementos.
4. El Transformer combina: atención, posición, residual, normalización y FFN.
5. Podemos aplicar la misma idea a texto, series temporales y visión por computador.

El modelo Transformer nos ayuda a aprender en general las relaciones entre elementos de una secuencia.

Gracias por vuestra atención

¿Preguntas?

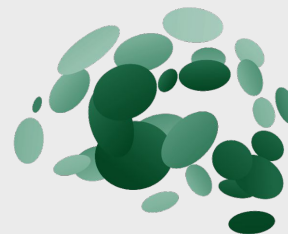


Ignacio Aguilera Martos

nacheteam@ugr.es



**UNIVERSIDAD
DE GRANADA**



DaSCI

Instituto Andaluz Interuniversitario en
Data Science and Computational Intelligence